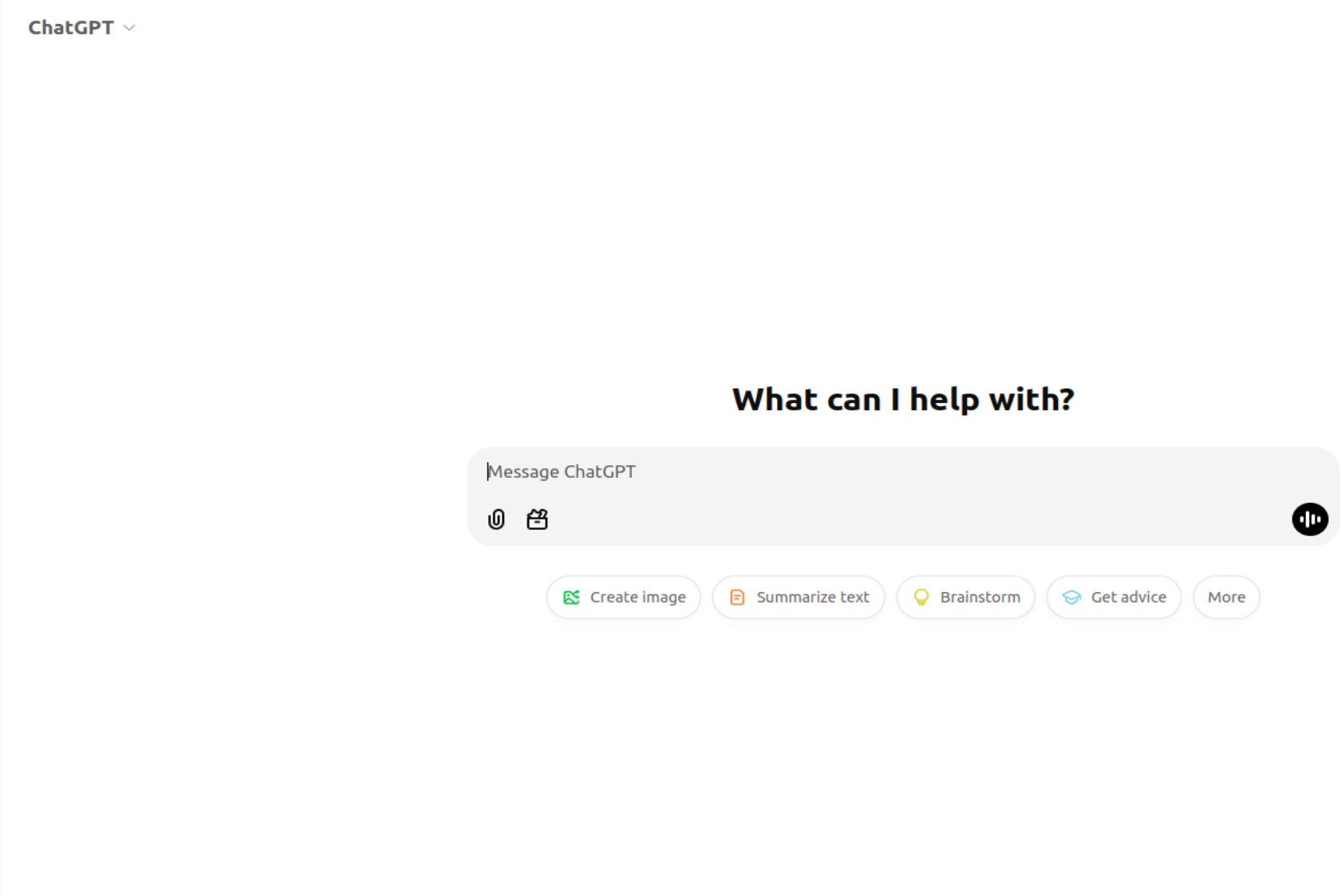
Python Programming

Ranjeet Ranjan Jha

December 14, 2024

ChatGPT


ChatGPT
What can I help with?
Message ChatGPT
Create image
Summarize text
Brainstorm
Get advice
More

History of Python

- Developed by Guido van Rossum in the late 1980s and early 1990s at the National Research Institute for Mathematics and Computer Science in the Netherlands
- Derived from languages including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell scripting languages
- Python source code is available under the GNU General Public License (GPL)
- The name Python is inspired by the comedy series Monty Python's Flying Circus

Major Python Releases

- **Python 0.9.0** (February 1991): First published version, supported core object-oriented programming principles
- **Python 1.0** (January 1994): Added functional programming tools, support for complex numbers
- **Python 2.0** (October 2000): Introduced list comprehensions, garbage collection, and Unicode support
- **Python 3.0** (December 2008): Revamped version to remove discrepancies in Python 2.x, backported to Python 2.6, included python2to3 utility for code translation
- **EOL for Python 2.x** (2020): Support for Python 2 discontinued, with 2.7.17 as the last version
- **Python 3.11.2** (February 2023): stable version, with significant speed improvements and enhanced exception messaging
- **Python 3.12.4...**

Characteristics of Python

- Interpreted: Processed at runtime by the interpreter, no need to compile
- Interactive: Direct interaction with the interpreter
- Object-Oriented: Supports encapsulating code within objects
- Beginner's Language: Suitable for developing a wide range of applications
- Open-source and cross-platform: Available on Linux, Windows, and Mac OS under the Python Software Foundation License

Why Learn Python?

- Consistently popular and easy to learn
- Open source and free
- Versatile: suitable for web development, AI, ML, automation, etc.
- Powerful libraries for AI, ML, etc.
- High demand and lucrative salaries in the job market
- Ideal for students and professionals aiming for software engineering

Python Jobs

- High demand in web development, AI, ML, Data Science, etc.
- Companies: Google, Intel, PayPal, Facebook, IBM, Amazon, Netflix, Pinterest, Uber, and more
- Career roles: Python developer, Full-stack developer, Machine learning engineer, Data scientist, Data analyst, DevOps engineer, Software engineer, etc.

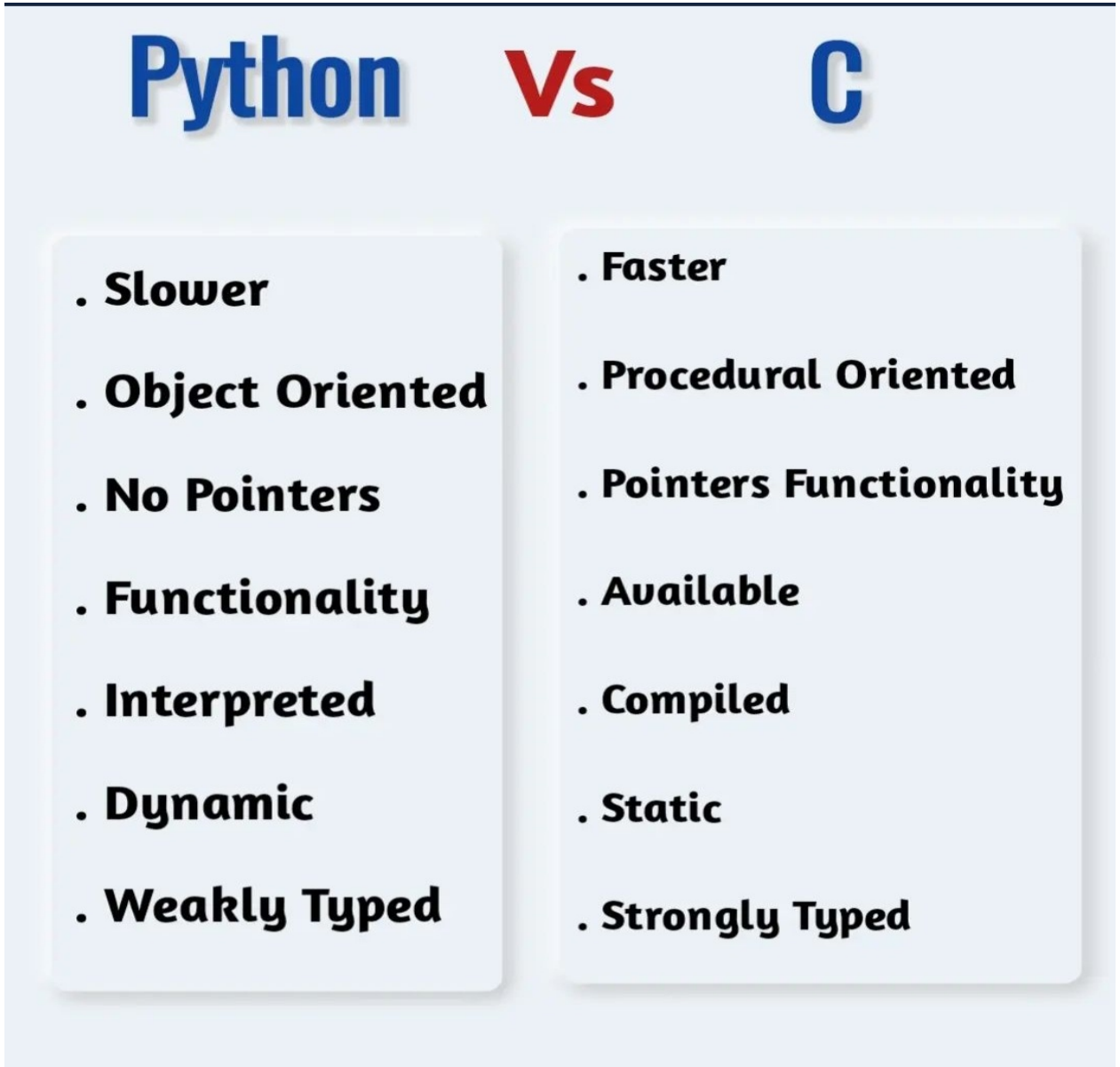
Applications of Python

- Data Science: Numpy, Pandas, Matplotlib
- Web Development: Django, Pyramid
- Computer Vision and Image Processing
- Automation, Job Scheduling, GUI Development

Features of Python

- Easy-to-learn: few keywords, simple structure, clear syntax
- Easy-to-read and maintain
- Broad standard library: portable and cross-platform
- Interactive Mode: supports interactive testing and debugging
- Portable: runs on various hardware platforms with the same interface
- Extendable: allows adding low-level modules to the interpreter
- Database interfaces: compatible with major commercial databases
- GUI Programming: supports GUI applications
- Scalable: better structure and support for large programs

C vs Python


Python Vs C
. Slower
. Object Oriented
. No Pointers
. Functionality
. Interpreted
. Dynamic
. Weakly Typed
. Faster
. Procedural Oriented
. Pointers Functionality
. Available
. Compiled
. Static
. Strongly Typed

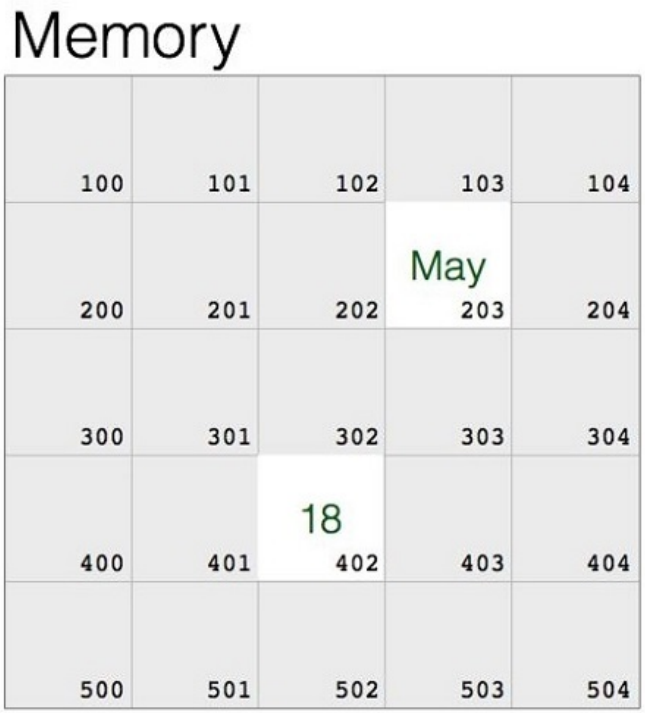
Python Compiler

- Provides examples to explain concepts
- Includes a Python compiler/interpreter

```
print("Hello, World!")
```

Python Variables

- Python variables are the reserved memory locations used to store values with in a Python Program.
- This means that when we create a variable you reserve some space in the memory.
- Memory Addresses



Python Variables

```
>>> "May"
May
>>> id("May")
2167264641264
>>> 18
18
>>> id(18)
140714055169352
```

- Once the data is stored in the memory, it should be accessed repeatedly for performing a certain process.
- Obviously, fetching the data from its ID is cumbersome.
- High level languages like Python make it possible to give a suitable alias or a label to refer to the memory location.

Python Variables

```
>>> "May"
May
>>> id("May")
2167264641264
>>> 18
18
>>> id(18)
140714055169352
```

- Once the data is stored in the memory, it should be accessed repeatedly for performing a certain process.
- Obviously, fetching the data from its ID is cumbersome.
- High level languages like Python make it possible to give a suitable alias or a label to refer to the memory location.

Python Variables

```
>>> month="May"
>>> age=18
```

```
>>> id(month)
2167264641264
>>> id(age)
140714055169352
```

Python Variables

```
counter = 100 # Creates an integer variable
miles = 1000.0 # Creates a floating point variable
name = "Zara Ali" # Creates a string variable
print (counter)
print (miles)
print (name)
```

Deleting Python Variables

```
del var
del var_a, var_b
```

```
counter = 100
print (counter)
del counter
print (counter)
```

```
100
Traceback (most recent call last):
File "main.py", line 7, in <module>
print (counter)
NameError: name 'counter' is not defined
```

Getting Type of a Variable

```
x = "Zara"
y = 10
z = 10.10
print(type(x))
print(type(y))
print(type(z))
```

This will produce the following result:

```
<class 'str'>
<class 'int'>
<class 'float'>
```

Casting Python Variables

```
x = str(10) # x will be '10'
y = int(10) # y will be 10
z = float(10) # z will be 10.0
print( "x =", x )
print( "y =", y )
print( "z =", z )
```

This will produce the following result:

```
x = 10
y = 10
z = 10.0
```

Case-Sensitivity of Python Variables

```
age = 20
Age = 30
print( "age =", age )
print( "Age =", Age )
```

This will produce the following result:

```
age = 20
Age = 30
```

Python Variables - Multiple Assignment

```
>>> a=10
>>> b=10
>>> c=10
```

Instead of separate assignments, you can do it in a single assignment statement as follows

```
>>> a=b=c=10
>>> print (a,b,c)
10 10 10
```

Python Variables - Multiple Assignment

```
>>> a=10
>>> b=20
>>> c=30
```

These separate assignment statements can be combined in one. You need to give comma separated variable names on left, and comma separated values on the right of = operator.

```
>>> print (a,b,c)
10 20 30
```

Python Variables - Multiple Assignment

```
a = b = c = 100
print (a)
print (b)
print (c)
```

```
a,b,c = 1,2,"Zara Ali"
print (a)
print (b)
print (c)
```

Python Variables - Multiple Assignment

```
a = b = c = 100
print (a)
print (b)
print (c)
```

```
a,b,c = 1,2,"Zara Ali"
print (a)
print (b)
print (c)
```

Python Variables - Naming Convention

- A variable name must start with a letter or the underscore character A variable name cannot start with a number or any special character like $, (, * % etc.
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Python variable names are case-sensitive which means Name and NAME are two different variables in Python.
- Python reserved keywords cannot be used naming the variable.

Python Variables - Naming Convention

If the name of variable contains multiple words, we should use these naming patterns -

- **Camel case** - First letter is a lowercase, but first letter of each subsequent word is in uppercase. For example: kmPerHour, pricePerLitre
- **Pascal case** - First letter of each word is in uppercase. For example: KmPerHour, PricePerLitre.
- **Snake case** - Use single underscore (_) character to separate words. For example: km_per_hour, price_per_litre

Python Variables - Naming Convention

```
counter = 100
_count = 100
name1 = "Zara"
name2 = "Nuha"
Age = 20
zara_salary = 100000
print (counter)
print (_count)
print (name1)
print (name2)
print (Age)
print (zara_salary)
```

Python Variables - Naming Convention

```
1counter = 100
$count = 100
zara-salary = 100000
print (1counter)
print ($count)
print (zara-salary)
```

Python Variables

```
>>> width=10
>>> height=20
>>> area=width*height
>>> area
200
>>> perimeter=2*(width+height)
>>> perimeter # 60
```

```
width = 10
height = 20
area = width*height
perimeter = 2*(width+height)
print ("Area = ", area)
print ("Perimeter = ", perimeter)
```

Python Local Variables

Python Local Variables are defined inside a function. We can not access variable outside the function.

```
def sum(x, y):
    sum = x + y
    return sum
print(sum(5, 10))
```

Python Global Variables

Any variable created outside a function can be accessed within any function and so they have global scope.

```
x = 5
y = 10
def sum():
    sum = x + y
    return sum
print(sum())
```

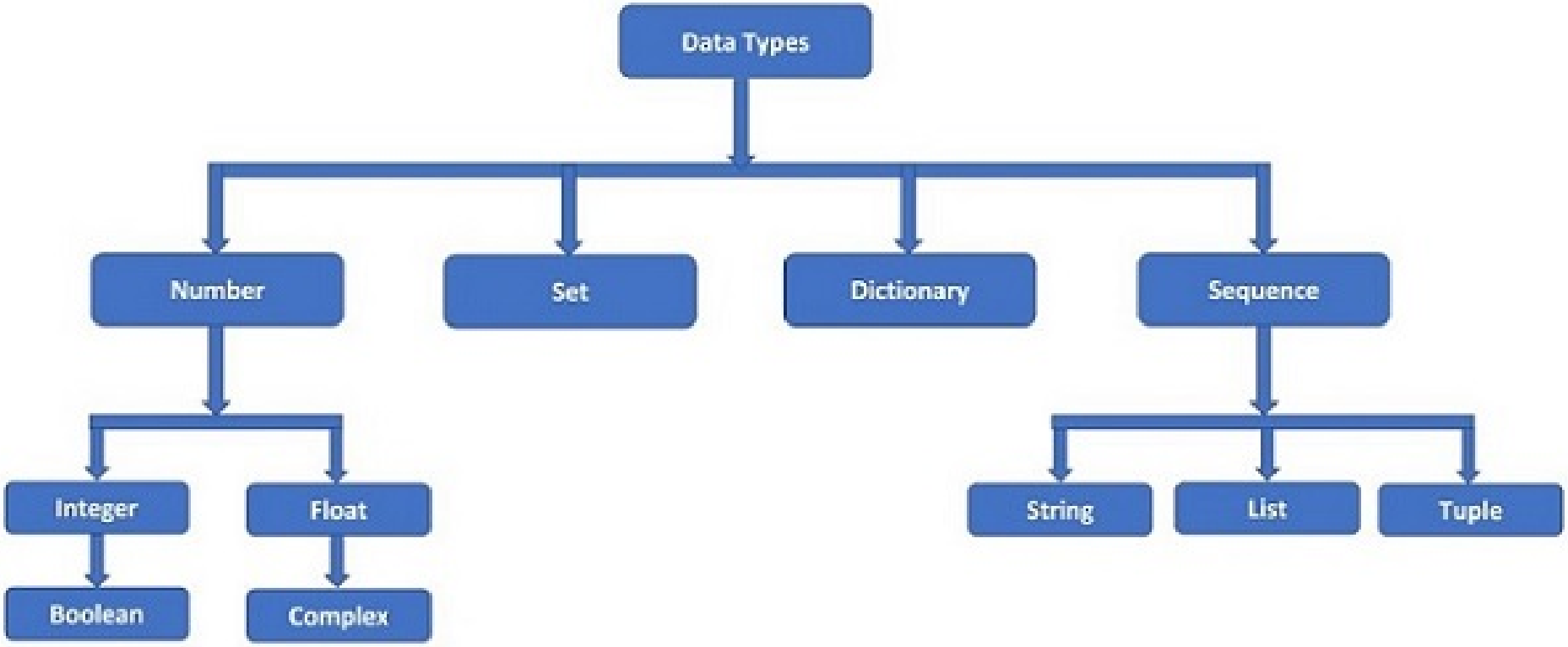
Python Variables

```
>>> a=b=10
>>> a is b
True
>>> id(a), id(b)
(140731955278920, 140731955278920)
```

Constants in Python

- Python doesn't have any formally defined constants, However we can indicate a variable to be treated as a constant by using all-caps names with underscores.
- For example, the name PI_VALUE indicates that you don't want the variable redefined or changed in any way.

Types of Python Data Types



Python Data Types

Python Numeric Data Types

```
var1 = 1 # int data type
var2 = True # bool data type
var3 = 10.023 # float data type
var4 = 10+3j # complex data type
```

Python Data Types

Python String Data Type

```
str = 'Hello World!'
print (str) # Prints complete string
print (str[0]) # Prints first character of the string
print (str[2:5]) # Prints characters starting from 3rd to 5th

print (str[2:]) # Prints string starting from 3rd character
print (str * 2) # Prints string two times
print (str + "TEST") # Prints concatenated string
```

Python List Data Type

```
list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]
tinylist = [123, 'john']
print (list) # Prints complete list
print (list[0]) # Prints first element of the list
print (list[1:3]) # Prints elements starting from 2nd till 3rd

print (list[2:]) # Prints elements starting from 3rd element

print (tinylist * 2) # Prints list two times
print (list + tinylist) # Prints concatenated lists
```

Python Tuple Data Type

```
tuple = ( 'abcd', 786 , 2.23, 'john', 70.2 )
tinytuple = (123, 'john')
print (tuple) # Prints the complete tuple
print (tuple[0]) # Prints first element of the tuple
print (tuple[1:3]) # Prints elements of the tuple starting fro
2nd till 3rd
print (tuple[2:]) # Prints elements of the tuple starting from
3rd element
print (tinytuple * 2) # Prints the contents of the tuple twice

print (tuple + tinytuple) # Prints concatenated tuples
```

List/Tuple

```
tuple = ('abcd', 786 , 2.23, 'john', 70.2 )
list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]
tuple[2] = 1000 # Invalid syntax with tuple
list[2] = 1000 # Valid syntax with list
```

Python Dictionary Data Type

```
dict = {}
dict['one'] = "This is one"
dict[2] = "This is two"
tinydict = {'name': 'john','code':6734, 'dept': 'sales'}

print (dict['one']) # Prints value for 'one' key
print (dict[2]) # Prints value for 2 key
print (tinydict) # Prints complete dictionary
print (tinydict.keys()) # Prints all the keys
print (tinydict.values()) # Prints all the values
```

Python Set Data Type

```
set1 = {123, 452, 5, 6}
set2 = {'Java', 'Python', 'JavaScript'}
print(set1)
print(set2)
```

Python Boolean Data Type

```
>>> type (True)
<class 'bool'>
>>> type(False)
<class 'bool'>

# Returns false as a is not equal to b
a = 2
b = 4
print(bool(a==b))
# Following also prints the same
print(a==b)
# Returns False as a is None
a = None
print(bool(a))
```

Python Boolean Data Type

```
# Returns false as a is an empty sequence
a = ()
print(bool(a))
# Returns false as a is 0
a = 0.0
print(bool(a))
```

Python Data Type Conversion

```
print("Conversion to integer data type")
a = int(1)  a will be 1
b = int(2.2)  b will be 2
c = int("3.3")  c will be 3

print (a)
print (b)
print (c)
print("Conversion to floating point number")
a = float(1)  a will be 1.0
b = float(2.2)  b will be 2.2
c = float("3.3")  c will be 3.3
print (a)
print (b)
print (c)
```

Python Data Type Conversion

```
print("Conversion to string")
a = str(1)  a will be "1"
b = str(2.2)  b will be "2.2"
c = str("3.3")  c will be "3.3"

print (a)
print (b)
print (c)
```

Python Type Casting

Implicit

```
<<< a=10  int object
<<< b=10.5  float object
<<< c=a+b
<<< print (c)
```

```
a=True
b=10.5
c=a+b
print (c)
```

Python Type Casting

Explicit

```
<<< a = int("100")
<<< a
100
<<< type(a)
<class 'int'>
<<< a = ("10"+"01")
<<< a = int("10"+"01")
<<< a
1001
<<< type(a)
<class 'int'>
```

Python Type Casting

Explicit

```
<<< a = int(10.5) converts a float object to int
<<< a
10
<<< a = int(2*3.14) expression results float, is converted to

int
<<< a
6
<<< type(a)
<class 'int'>
```

Python Type Casting

Explicit

```
<<< a = int("10.5")
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '10.5'
<<< a = int("Hello World")
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'Hello Wor
```

Conversion of Sequence Types

```
<<< a=[1,2,3,4,5]  List Object
<<< b=(1,2,3,4,5)  Tupple Object
<<< c="Hello"  String Object
### list() separates each character in the string and builds
the list
<<< obj=list(c)
<<< obj
['H', 'e', 'l', 'l', 'o']
### The parentheses of tuple are replaced by square brackets

<<< obj=list(b)
<<< obj
[1, 2, 3, 4, 5]
```

Conversion of Sequence Types

```
### tuple() separates each character from string and builds a
tuple of characters
<<< obj=tuple(c)
<<< obj
('H', 'e', 'l', 'l', 'o')
### square brackets of list are replaced by parentheses.
<<< obj=tuple(a)
<<< obj
(1, 2, 3, 4, 5)
```

Conversion of Sequence Types

```
### str() function puts the list and tuple inside the quote sy
<<< obj=str(a)
<<< obj
'[1, 2, 3, 4, 5]'
<<< obj=str(b)
<<< obj
'(1, 2, 3, 4, 5)'
```

Types of Python Operators

```
a = 21 b = 10 c = 0
c = a + b
print ("a: {} b: {} a+b: {}".format(a,b,c))
c = a - b
print ("a: {} b: {} a-b: {}".format(a,b,c) )
c = a * b
print ("a: {} b: {} a*b: {}".format(a,b,c))
c = a / b
print ("a: {} b: {} a/b: {}".format(a,b,c))
c = a
print ("a: {} b: {} a%b: {}".format(a,b,c))
```

Types of Python Operators

```
a = 2
b = 3
c = a**b
print ("a: {} b: {} a**b: {}".format(a,b,c))
a = 10
b = 5
c = a//b
print ("a: {} b: {} a//b: {}".format(a,b,c))
```

Precedence and Associativity of Python Arithmetic Operators

Operator(s)	Description	Associativity
**	Exponent	Right to Left
*, /, //, %	Multiplication, Division, Floor Division, Modulus	Left to Right
+, -	Addition, Subtraction	Left to Right

Table: Precedence and Associativity of Python Arithmetic Operators

Complex Number Arithmetic

```
a=2.5+3.4j
b=-3+1.0j
print ("Addition of complex numbers - a=",a, "b=",b, "a+b=",
a+b)
print ("Subtraction of complex numbers - a=",a, "b=", b, "a-b=
a-b)

a=6+4j
b=3+2j
c=a*b
c=(18-8)+(12+12)j
c=10+24j
```

Python Comparison Operators

Operator	Description	Example
<	Less than	a < b
>	Greater than	a > b
<=	Less than or equal to	a <= b
>=	Greater than or equal to	a >= b
==	Is equal to	a == b
!=	Is not equal to	a != b

Table: Comparison Operators in Python

Python Comparison Operators

```
a=5 b=7 print (a>b)
print (a<b)
print ("Both operands are integer")
a=5
b=7
print ("a=",a, "b=",b, "a>b is", a>b)
print ("a=",a, "b=",b,"a<b is",a<b)
print ("a=",a, "b=",b,"a==b is",a==b)
print ("a=",a, "b=",b,"a!=b is",a!=b)
```

Comparison of Float Number

```
print ("comparison of int and float")
a=10
b=10.0
print ("a=",a, "b=",b, "a>b is", a>b)
print ("a=",a, "b=",b,"a<b is",a<b)
print ("a=",a, "b=",b,"a==b is",a==b)
print ("a=",a, "b=",b,"a!=b is",a!=b)
```

Comparison of Sequence Types

In Python, comparison of only similar sequence objects can be performed.
A string object is comparable with another string only.
A list cannot be compared with a tuple, even if both have same items.

```
print ("comparison of tuples")
a=(1,2,4)
b=(1,2,3)
print ("a=",a, "b=",b,"a<b is",a<b)
print ("a=",a, "b=",b,"a>b is",a>b)
print ("a=",a, "b=",b,"a==b is",a==b)
print ("a=",a, "b=",b,"a!=b is",a!=b)
```

Comparison of Sequence Types

```
print ("comparison of different sequence types")
a=(1,2,3)
b=[1,2,3]
print ("a=",a, "b=",b,"a<b is",a<b)
```

Python Assignment Operator

The = (equal to) symbol is defined as assignment operator in Python. The value of Python expression on its right is assigned to a single variable on its left.

```
a = 10
b = 5
a = a + b
print (a)
print ("Augmented addition of int and int")
a+=b  equivalent to a=a+b
print ("a=",a, "type(a):", type(a))

print ("Augmented subtraction of int and int")
a-=b equivalent to a=a-b
print ("a=",a, "type(a):", type(a))
```

Python Logical Operators

Python logical operators are used to form compound Boolean expressions.

a	b	a and b
F	F	F
F	T	F
T	F	F
T	T	T

Table: Truth Table for Logical "and" Operator

a	b	a or b
F	F	F
F	T	T
T	F	T
T	T	T

Table: Truth Table for Logical "or" Operator

Python Logical Operators

a	not (a)
F	T
T	F

Table: Truth Table for Logical "not" Operator

```
x = 10
y = 20
print("x > 0 and x < 10:",x > 0 and x < 10)
print("x > 0 and y > 10:",x > 0 and y > 10)
print("x > 10 or y > 10:",x > 10 or y > 10)
print("x%2 == 0 and y%2 == 0:",x%2 == 0 and y%2 == 0)
print ("not (x+y>15):", not (x+y)>15)
```

Python - Python Membership Operators

The 'in' Operator

```
var = "Something"
a = "S"
b = "om"
c = "thing"
d = "ng"
print (a, "in", var, ":", a in var)
print (b, "in", var, ":", b in var)
print (c, "in", var, ":", c in var)
print (d, "in", var, ":", d in var)
```

Python - Python Membership Operators

The 'not in' Operator

```
var = "Something"
a = "S"
b = "om"
c = "thing"
d = "ng"
print (a, "not in", var, ":", a not in var)
print (b, "not in", var, ":", b not in var)
print (c, "not in", var, ":", c not in var)
print (d, "not in", var, ":", d not in var)
```

Python - Python Membership Operators

```
var = [10,20,30,40]
a = 20
b = 10
c = a-b
d = a/2
print (a, "in", var, ":", a in var)
print (b, "not in", var, ":", b not in var)
print (c, "in", var, ":", c in var)
print (d, "not in", var, ":", d not in var)
```

Python - Python Membership Operators

```
var = (10,20,30,40)
a = 10
b = 20
print ((a,b), "in", var, ":", (a,b) in var)
var = ((10,20),30,40)
a = 10
b = 20
print ((a,b), "in", var, ":", (a,b) in var)
```

Python - Python Membership Operators

Membership Operator with Sets

```
var = {10,20,30,40}
a = 10
b = 20
print (b, "in", var, ":", b in var)
var = {(10,20),30,40}
a = 10
b = 20
print ((a,b), "in", var, ":", (a,b) in var)
```

Python - Python Membership Operators

Membership Operator with Dictionaries:
Python checks the membership only with the collection of keys and not values.

```
var = 1:10, 2:20, 3:30
a = 2
b = 20
print (a, "in", var, ":", a in var)
print (b, "in", var, ":", b in var)
```

Python - Comments

```
# Standalone single line comment is placed here

# This function calculates the factorial of a number
# using an iterative approach. The factorial of a number
# n is the product of all positive integers less than or
# equal to n. For example, factorial(5) is 5*4*3*2*1 = 120.

""" This function calculates the greatest common divisor (GCD)
of two numbers using the Euclidean algorithm. The GCD of two
numbers is the largest number that divides both of them withou
leaving a remainder. """
```

Python - Decision Making

Decision Making Statements: Decision making statements are used in the Python programs to make them able to decide which of the alternative group of instructions to be executed, depending on value of a certain Boolean expression.

```
marks = 80
result = ""
if marks < 30:
    result = "Failed"
elif marks > 75:
    result = "Passed with distinction"
else:
    result = "Passed"
print(result)
```

Python - Decision Making

```
var = 100
if ( var == 100 ) : print ("Value of expression is 100")
print ("Good bye!")
```

```
var = 100
if ( var == 100 ):
    print ("Value of var is equal to 100")
else:
    print("Value of var is not equal to 100")
```

Python - Decision Making

Nested if statements

```
num=8
print ("num = ",num)
if num%2==0:
    if num%3==0:
        print ("Divisible by 3 and 2")
    else:
        print ("divisible by 2 not divisible by 3")
else:
    if num%3==0:
        print ("divisible by 3 not divisible by 2")
    else:
        print ("not Divisible by 2 not divisible by 3")
```